

AI ASS 20.5

Name: B.Sanjana

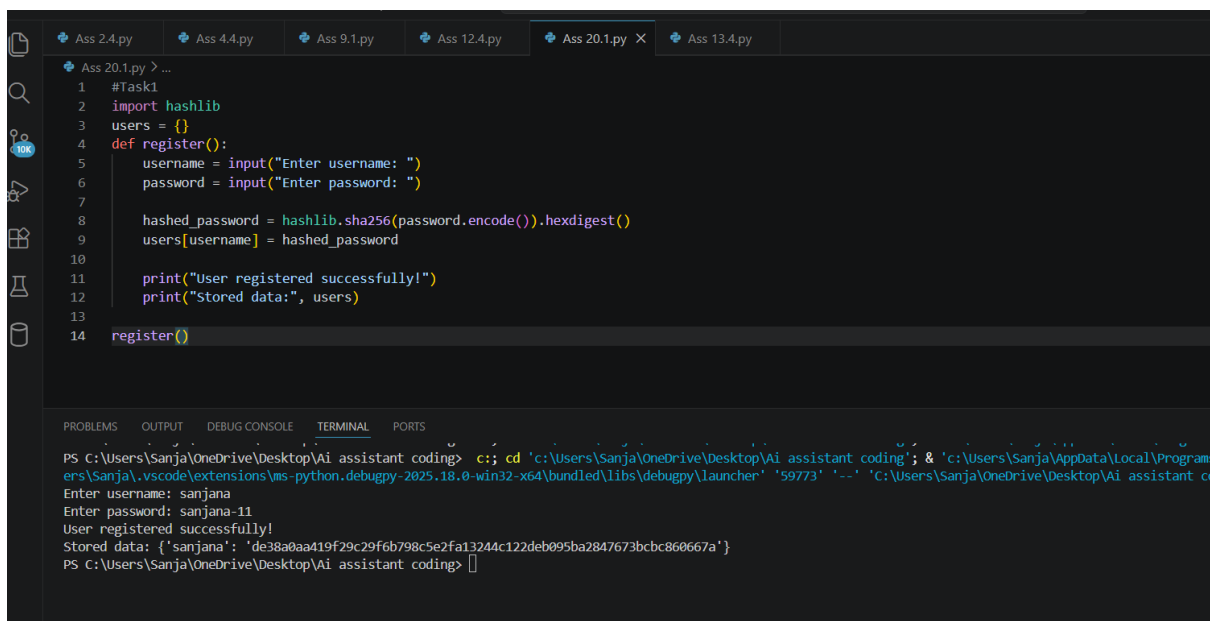
H.T.NO: 2303A52306

Batch: 43

#Task1

Prompt:

Generate a Python user registration program and improve it by securely storing passwords using hashing instead of plain text.



The screenshot shows a Visual Studio Code editor with a Python file named 'Ass 20.1.py'. The code implements a user registration function that uses the hashlib library to hash passwords with SHA-256. The terminal output shows the program being executed, with user input 'sanjana' and 'sanjana-11', and the resulting stored data: {'sanjana': 'de38a0aa419f29c29f6b798c5e2fa13244c122deb095ba2847673bcb860667a'}.

```
1 #Task1
2 import hashlib
3 users = {}
4 def register():
5     username = input("Enter username: ")
6     password = input("Enter password: ")
7
8     hashed_password = hashlib.sha256(password.encode()).hexdigest()
9     users[username] = hashed_password
10
11     print("User registered successfully!")
12     print("Stored data:", users)
13
14 register()
```

```
PS C:\Users\Sanja\OneDrive\Desktop\Ai assistant coding> c::; cd 'c:\Users\Sanja\OneDrive\Desktop\Ai assistant coding'; & 'c:\Users\Sanja\AppData\Local\Programs\Microsoft VS Code\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '59773' '--' 'c:\Users\Sanja\OneDrive\Desktop\Ai assistant coding\Ass 20.1.py'
Enter username: sanjana
Enter password: sanjana-11
User registered successfully!
Stored data: {'sanjana': 'de38a0aa419f29c29f6b798c5e2fa13244c122deb095ba2847673bcb860667a'}
PS C:\Users\Sanja\OneDrive\Desktop\Ai assistant coding>
```

Observation:

The vulnerability of plain-text password storage was identified and fixed by applying SHA-256 hashing.

This ensures secure password storage and protects user credentials from unauthorized access.

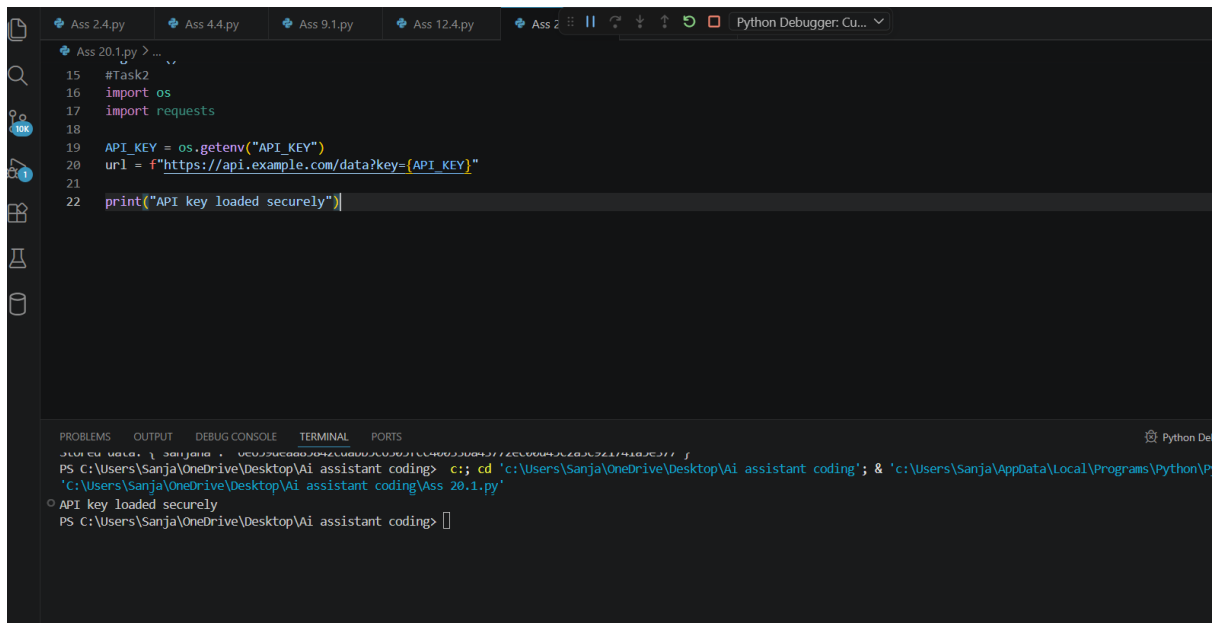
#Task2:

Prompt:

Generate a Python script to connect to an external API.

Check whether API keys are hardcoded in the code.

Refactor it using environment variables for secure secret management.



```
15 #Task2
16 import os
17 import requests
18
19 API_KEY = os.getenv("API_KEY")
20 url = f"https://api.example.com/data?key={API_KEY}"
21
22 print("API key loaded securely")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\Sanja\OneDrive\Desktop\Ai assistant coding> c:: cd 'c:\Users\Sanja\OneDrive\Desktop\Ai assistant coding'; & 'c:\Users\Sanja\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\Sanja\OneDrive\Desktop\Ai assistant coding\Ass 20.1.py'

API key loaded securely

PS C:\Users\Sanja\OneDrive\Desktop\Ai assistant coding>

Observation:

The original script contained a hardcoded API key, which is a security vulnerability.

Hardcoded secrets can be exposed if the code is shared publicly.

The issue was fixed using environment variables with `os.getenv()`.

This ensures secure and proper management of sensitive credentials.

#Task3:

Prompt:

Generate a Flask file upload example.

Check for unsafe file handling vulnerabilities.

Improve it using file type, size, and filename validation.

```
Ass 20.1.py > ...
23 #Task3
24 from flask import Flask, request, redirect
25 from werkzeug.utils import secure_filename
26 import os
27 app = Flask(__name__)
28 UPLOAD_FOLDER = 'uploads'
29 app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER
30 app.config['MAX_CONTENT_LENGTH'] = 2 * 1024 * 1024
31 ALLOWED_EXTENSIONS = {'png', 'jpg', 'jpeg', 'pdf'}
32 # Check allowed file
33 def allowed_file(filename):
34     return '.' in filename and filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS
35 # Home page (simple upload form)
36 @app.route('/')
37 def home():
38     return '''
39     <h2>Upload File</h2>
40     <form method="post" action="/upload" enctype="multipart/form-data">
41         <input type="file" name="file">
42         <input type="submit" value="Upload">
43     </form>
44     '''
45 # Upload route
46 @app.route('/upload', methods=['POST'])
47 def upload_file():
48     if 'file' not in request.files:
49         return "No file part"
50     file = request.files['file']
51     if file.filename == '':
52         return "No selected file"
53     if file and allowed_file(file.filename):
54         filename = secure_filename(file.filename) # Secure filename
55         file.save(os.path.join(app.config['UPLOAD_FOLDER'], filename))
56         return "File uploaded securely!"
57     return "Invalid file type!"
58 # Run app
59 if __name__ == '__main__':
60     # Create upload folder if not exists
61     if not os.path.exists(UPLOAD_FOLDER):
62         os.makedirs(UPLOAD_FOLDER)
63     app.run(debug=True)
```

Output:

← → ↺ 127.0.0.1:5000

Upload File

my resume.pdf



Observation:

The original file upload script was vulnerable to unsafe file handling.

Validation was added to allow only specific file types such as png, jpg, and pdf.

A file size limit and `secure_filename()` were used to prevent malicious uploads and unsafe filenames.

This ensures a secure file upload implementation resistant to common file-based attacks.

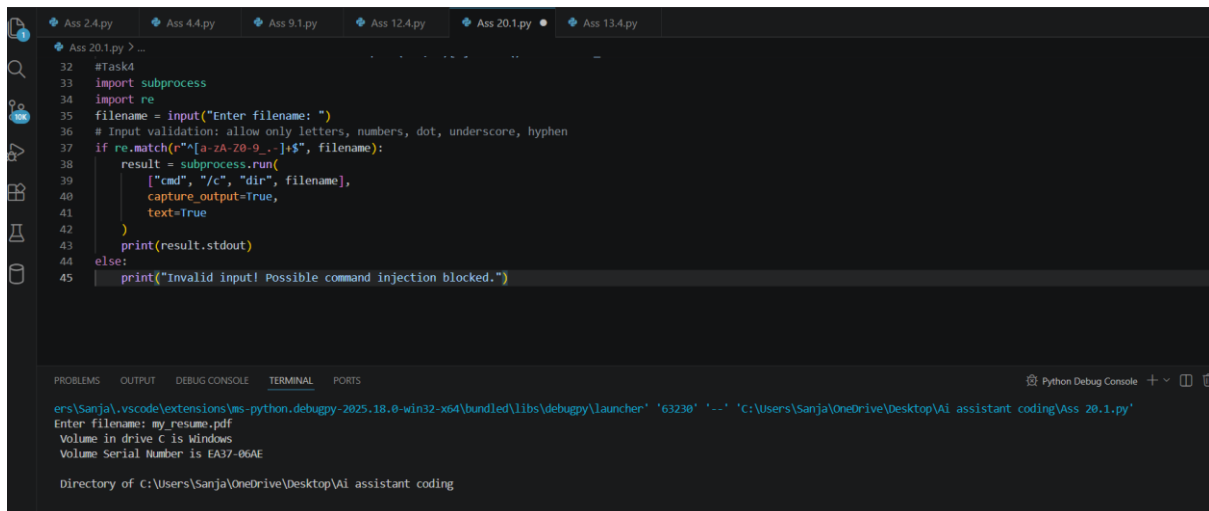
#Task4:

Prompt:

Generate a Python script that executes system commands using `subprocess`.

Check for command injection risks with user input.

Improve it using safe `subprocess` calls and input validation.

The image shows a VS Code editor window with a Python file named 'Ass 20.1.py'. The script is a function called '#Task4' that takes a filename as input. It uses 'import subprocess' and 'import re'. It prompts the user to 'Enter filename:'. It then uses 're.match' with a regular expression '^[a-zA-Z0-9_\\.]+\$' to validate the input. If the input is valid, it runs 'subprocess.run' with a list of arguments: ['cmd', '/c', 'dir', filename]. It sets 'capture_output=True' and 'text=True'. It then prints 'result.stdout'. If the input is invalid, it prints 'Invalid input! Possible command injection blocked.' The terminal at the bottom shows the execution of the script. It displays the command prompt path, the prompt 'Enter filename: my_resume.pdf', and the output 'Volume in drive C is Windows' and 'Volume Serial Number is EA37-86AE'. It also shows the directory path 'C:\Users\Sanja\OneDrive\Desktop\Ai assistant coding'.

Observation:

The original script using `os.system()` was vulnerable to command injection through unsafe user input.

The issue was fixed by replacing it with `subprocess.run()` using a list of arguments.

Input validation was added using regular expressions to allow only safe characters.

This prevents execution of malicious commands and ensures secure command handling.

#Task5:

Prompt:

Generate an HTML and Flask feedback form.

Check for missing input validation and unsafe rendering.

Improve it using server-side validation and sanitization.

```
Ass 2.4.py  Ass 4.4.py  Ass 9.1.py  Ass 12.4.py  Ass 20.1.py X
Ass 20.1.py > ...
46 #Task5
47 from flask import Flask, request
48 import html
49
50 app = Flask(__name__)
51
52 @app.route("/", methods=["GET", "POST"])
53 def feedback():
54     if request.method == "POST":
55         name = html.escape(request.form["name"])
56         message = html.escape(request.form["message"])
57
58         if len(name) < 3:
59             return "Invalid name"
60
61         return f"Thank you, {name}! Feedback received safely."
62
63     return '''
64     <form method="POST">
65         Name: <input type="text" name="name"><br><br>
66         Feedback: <textarea name="message"></textarea><br><br>
67         <input type="submit" value="Submit">
68     </form>
69     '''
70
71 if __name__ == "__main__":
72     app.run(debug=True)
```

Output:

The screenshot shows a web browser at 127.0.0.1:5000. The first part shows the feedback form with 'Name' set to 'sanjana' and 'Feedback' set to 'very useful application'. A 'Submit' button is visible. The second part shows the browser displaying the message: 'Thank you, sanjana! Feedback received safely.'

Observation:

The original web form lacked proper server-side validation and sanitization.

This could lead to injection attacks such as XSS and unsafe rendering.

The issue was fixed using HTML escaping and input length validation.

This makes the form secure and resistant to malicious input attacks.